

ShopFlow

Plateforme data e-commerce sur Snowflake

Ingestion → Transformation automatisée → Analytics

AUTEUR

Abraham Koloboé

Projet fil rouge • 5 jours
Snowflake / Data Engineering

Contexte

ShopFlow, jeune boutique e-commerce française spécialisée **high-tech**, génère chaque jour **4 sources de données**.



Commandes

CSV



Référentiel clients

Parquet



Catalogue produits

JSON



Événements web

JSON semi-structuré



PROBLÈME

Ces données « dorment » dans un bucket.



OBJECTIF MÉTIER

Les exploiter pour piloter le business.

Objectifs du projet

1 Architecture multicouche

RAW → STAGING → MARTS

2 Virtual warehouses dimensionnés

selon les workloads

3 Ingestion mixte

structuré + semi-structuré

4 External tables

interrogation directe des fichiers

5 Pipeline automatisé

Streams · Tasks · Dynamic Tables

6 Time Travel

récupération & historique

7 Dashboard de restitution

pilotage métier

7 objectifs · une chaîne data de bout en bout

Données à disposition

LOT INITIAL · J1 – PREMIER CHARGEMENT

VOLUME

customers.parquet	Clients	Parquet	~10 000
products.json	Catalogue produits	JSON (array)	~500
orders.csv	Commandes	CSV	~50 000
order_items.csv	Lignes de commande	CSV	~150 000
web_events.json	Événements web	NDJSON	~200 000



LOT J2 · LE « LENDEMAIN »

orders · order_items · web_events rejoués — prouvent que le pipeline capte **automatiquement** les nouvelles données.

Particularités des données

T IDs alphanumériques

00000001 · C005654 · P00085 → VARCHAR

Surtout pas NUMBER : les zéros de tête seraient perdus.

[] JSON en array

[{...}, {...}] → STRIP_OUTER_ARRAY

products.json est un tableau, pas un objet par ligne.

🕒 Dates horodatées

order_date = ... 14:32:07 → TIMESTAMP

order_date contient l'heure — pas un simple DATE.

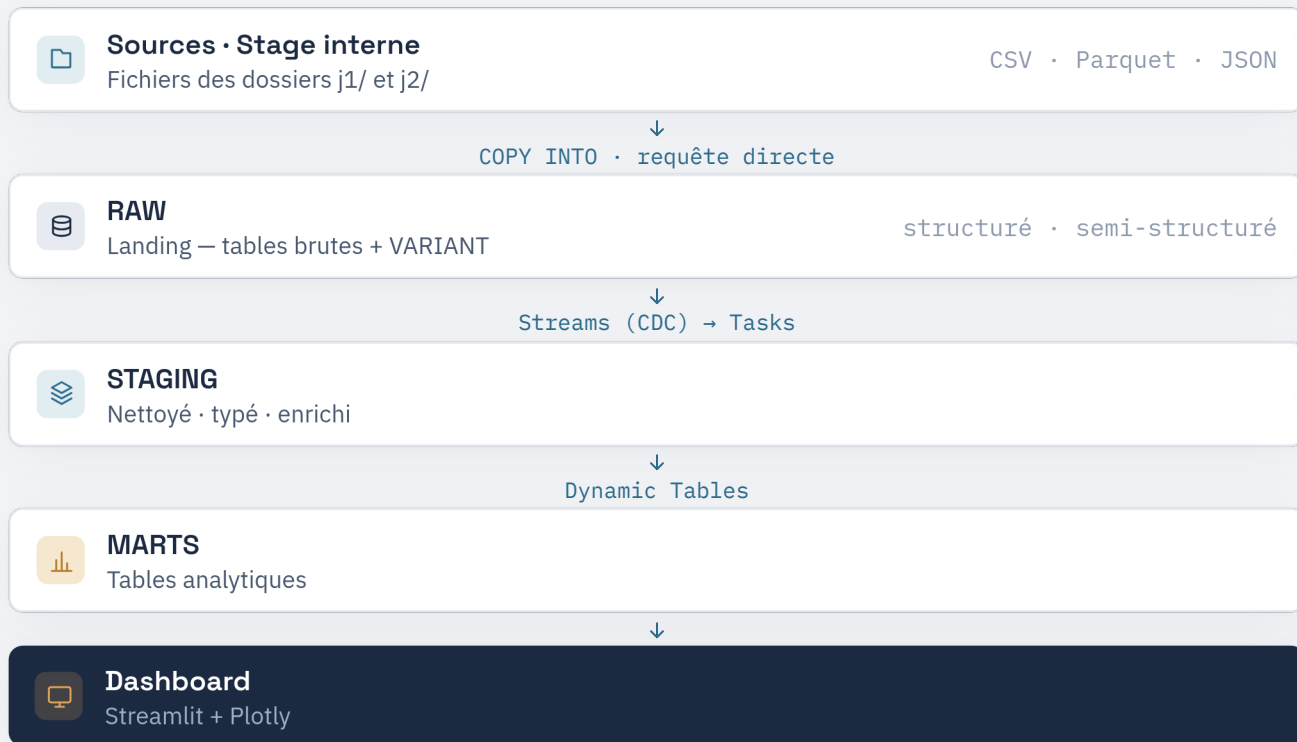
📄 Objet imbriqué & nulls

context{...} · user_id? → VARIANT + nulls gérés

user_id, product_id, referrer nullables.

💡 **Leçon** — toujours prévisualiser les fichiers avant de figer un schéma.

Architecture cible



COMPUTE · 2 WAREHOUSES

WH_INGEST

Chargements (Sources → RAW)

XSMALL

WH_TRANSFORM

Transformations (RAW → MARTS)

SMALL

Séparer le compute **isole les workloads** et maîtrise les coûts.

Infrastructure & gouvernance

Stockage & schémas

Base

SHOPFLOW_DB

RAW

STAGING

MARTS

Stage interne — dossiers j1/ et j2/

3 file formats — CSV · Parquet · JSON

Compute

WH_INGEST

XSMALL

WH_TRANSFORM

SMALL

 Auto-suspend **60 s**

Warehouses dédiés — pas de contention.

Gouvernance & coûts

Rôle applicatif

SHOPFLOW_ENGINEER

+ EXECUTE TASK

- Warehouses dédiés & auto-suspend
- Suspension manuelle en fin de journée

Couche RAW

zone de landing



Structuré

[COPY INTO](#)

ORDERS_RAW

ORDER_ITEMS_RAW

CUSTOMERS_RAW

+ colonnes d'audit (source, horodatage de chargement)



Semi-structuré

[VARIANT](#)

PRODUCTS_RAW

WEB_EVENTS_RAW

Requêtés via data : champ : :type et LATERAL FLATTEN



LIMITE SNOWFLAKE

External table impossible sur un **stage interne**.



CONTOURNEMENT

Pattern data-lake — requête directe sur fichier + benchmark **stage vs table matérialisée**.

Pipeline automatisé

STAGING

🔗 GRAPHE DE 4 TASKS

TSK_LOAD_STG_PRODUCTS

racine · 5 min · full refresh

TSK_LOAD_STG_ORDERS



TSK_LOAD_STG_ORDER_ITEMS

enrichissement produit

TSK_LOAD_STG_WEB_EVENTS

parsing VARIANT

STREAMS · 3 APPEND-ONLY

STR_ORDERS

STR_ORDER_ITEMS

STR_WEB_EVENTS

✓ Backfill initial **J1** + incrémental **J2** via streams — pas de double-comptage.

🕒 **WHEN SYSTEM\$STREAM_HAS_DATA** — des crédits **seulement** s'il y a des données.

NETTOYAGE

nulls traités

dates typées

VARIANT éclaté

LINE_AMOUNT calculé

Couche MARTS

↻ 3 Dynamic Tables · target_lag 5 min · auto



DT_DAILY_REVENUE

- Chiffre d'affaires
- Nombre de commandes
- Panier moyen / jour

Refresh incrémental



DT_TOP_PRODUCTS

- Top 20 par chiffre d'affaires
- Fenêtre 30 jours
- Ancrée sur la **date max** des données

Full refresh



DT_CUSTOMER_COHORTS

- Rétention client
- Par mois d'inscription

Full refresh

Robustesse

Time Travel

SCÉNARIO

- 1 DELETE « accidentel »**
1 000 commandes supprimées de `ORDERS_RAW`
- 2 Restauration chirurgicale**
`BEFORE(STATEMENT) MINUS` état courant
- 3 Alternative**
`UNDROP` sur table supprimée

ISOLATION ENTRE COUCHES

RAW

× DELETE appliqué

STAGING

✓ intact

MARTS

✓ intact

Le DELETE sur RAW n'a jamais touché STAGING ni MARTS —
l'architecture multicouche prouve sa valeur.

Rendu final dashboard



Streamlit + Plotly

interactif · double mode SiS / local

⌵ FILTRES SIDEBAR

période

statut

catégories

appareil

granularité

top N

6 SECTIONS

01 KPIs

02 Évolution du CA

03 Produits

04 Comportement web

05 Rétention (cohortes)

06 Dernières commandes



Glissez ici la capture du dashboard

or [browse files](#)

Plus-value

AVANT

Données dormantes

APRÈS

Pilotage near-real-time**5 min**

de lag

**Restitution self-service & filtrable****Coûts maîtrisés by design****Robustesse & recovery****Séparation nette des couches**

traçabilité · maintenabilité

Forces, limites & améliorations

✓ Forces

- Architecture claire & séparée
- Pipeline incrémental automatisé
- Gouvernance des coûts
- Dashboard self-service
- Compute séparé (2 warehouses)
- Semi-structuré natif (VARIANT)
- Recovery Time Travel

⚠ Limites

- External table indispo (stage interne)
- Pas de tests qualité automatisés
- Environnement unique (pas de CI/CD)
- Append-only — pas de MERGE
- RGPD non couvert par défaut
- products / customers en full refresh

↗ ROADMAP — AMÉLIORATIONS

Stage externe S3/GCS + external tables + Snowpipe

MERGE (idempotence & updates)

Tests qualité (dbt / procédure + alertes)

Masking EMAIL + row access policy

CI/CD + environnements + IaC

Connexion BI (Power BI / Tableau)

Historisation SCD des clients

Monitoring coûts & observabilité des tasks

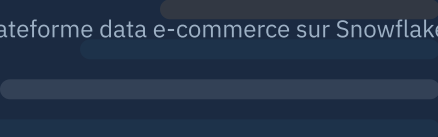
Merci



Questions ?

Abraham Koloboé

Plateforme data e-commerce sur Snowflake



Ingestion → Transformation → Analytics